

tmg-hmc: A Python package for Exact HMC Sampling for Truncated Multivariate Gaussians with Linear and Quadratic Constraints

Erik A. Bensen ¹ and Mikael Kuusela ¹

¹ Department of Statistics and Data Science, Carnegie Mellon University ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Matt Graham](#) 

Reviewers:

- [@dirmeier](#)
- [@AdrienCorenflos](#)

Submitted: 31 December 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Markov Chain Monte Carlo is a cornerstone of statistical methods that allows us to approximate intractable quantities by sampling from complex, multivariate probability distributions. One important class of distributions is constrained distributions. We present tmg-hmc: a Python implementation of the Exact Hamiltonian Monte Carlo for Truncated Multivariate Gaussians with linear and quadratic inequality constraints introduced in Pakman & Paninski (2014). This method leverages the high-dimensional scalability and good mixing properties of Hamiltonian Monte Carlo (HMC) while maintaining speed and simplicity since the Hamiltonian equations for a truncated Gaussian distribution are analytically solvable. This means that the sampler always accepts the sampled value and there are no tunable parameters. The original authors created an R implementation tmg and a Matlab implementation hmc_tmg. Both of these implementations are no longer maintained and the R package was archived from CRAN in 2021. Bertolacci et al. (2024) partially implements the Exact HMC in R, however, this implementation is limited to only linear constraints. There are also two R packages, VeccTMVN and nntmvn, that can sample truncated multivariate Gaussians. However, these implementations are both approximate and limited to linear box constraints. To the best of our knowledge, tmg-hmc is the only existing Python implementation of Exact HMC. Additionally, we expand our implementation by including sparse matrix operations for sparse constraint handling and optional GPU acceleration for high-dimensional problems such as truncated Gaussian processes. Finally, we accelerate the quadratic constraint hit-time calculation by using a speed-optimized C++ implementation that can be called from Python.

Statement of need

Many statistical models of real-world phenomena require the computation of intractable integrals over complex, multivariate probability distributions. Markov Chain Monte Carlo is a foundational statistical method that can be used to approximate these quantities from samples (Robert & Casella, 1999). This has allowed for significant progress in statistical modeling in many areas of applied statistics and machine learning (Gelman et al., 2013). One important class of distributions that arises due to parameter or data constraints are truncated distributions (Gelfand et al., 1992; Stanley et al., 2025; Swiler et al., 2020).

In Pakman & Paninski (2014), Exact HMC is used to sample from a d -dimensional Gaussian $X \sim N(\mu, \Sigma)$ that is truncated with m inequality constraints of the form

$$Q_j(X) \geq 0, \quad j = 1, \dots, m,$$

where $Q_j(X)$ is a product of linear and quadratic polynomials. As discussed by Pakman and Paninski, this type of a distribution is a critical component of a vast array of statistical models

including the probit and Tobit models (Albert & Chib, 1993; Tobin, 1958), the dichotomized Gaussian model (Cox & Wermuth, 2002; Emrich & Piedmonte, 1991), stochastic integrate-and-fire neural models (Paninski et al., 2003), Bayesian isotonic regression (Neelon & Dunson, 2004), and the Bayesian bridge model (Polson et al., 2014).

More recently, distributions of this form have been used for learning partially censored Gaussian processes (Cao & Katzfuss, 2025). The Exact HMC algorithm was used to implement physics-informed constraints for fields governing CO₂ flux in the WOMBAT v2.0 hierarchical flux-inversion framework for inferring changes to the global carbon cycle (Bertolacci et al., 2024). Additionally, sampling constrained multivariate normals is relevant to a growing range of literature on constrained Gaussian processes (Agrell, 2019; Bachoc et al., 2019, 2022; Da Veiga & Marrel, 2012; López-Lopera et al., 2018; Maatouk, 2017; Swiler et al., 2020).

Research Impact Statement

In our ongoing statistical research, we are using `tmg-hmc` to sample random transport maps given by the gradient of 2d convex Gaussian processes. We do this by approximating the convex GP by imposing quadratic convex inequality constraints on a discrete spatial grid. Figure 1 shows an example of such a transport map sampled using `tmg-hmc`. For context, one constraint is positivity of the second derivative for 416 coordinates. If we were to naively rejection sample with just these positivity constraints for a GP with small length scales so that the locations are approximately independent, the acceptance probability would approach $1/2^{416}$ which is completely infeasible. We also tested sampling by using a linearly constrained region as a proposal distribution and then rejection sampling to get to the convexity constraint, in this case we get an acceptance probability of $\approx 1/10^4$ which is improved over naive rejection sampling, but still infeasible for most applications. `tmg-hmc` allows us to sample directly from the convex Gaussian process distribution and we are able to sample several thousand samples per second.

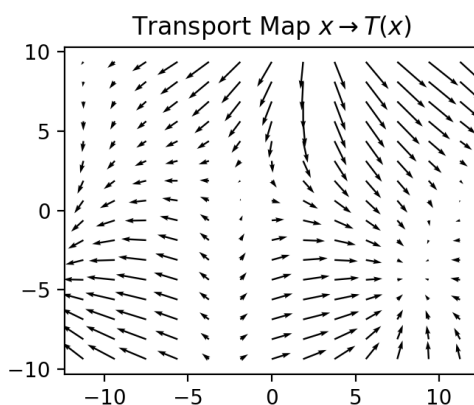


Figure 1: Sample of a random transport map defined as the gradient of a 2d convex Gaussian process.

Preliminary work, including our use of `tmg-hmc`, has already been presented at several scientific conferences and has drawn considerable interest from the statistical and geoscientific communities. Additionally, this work on random transport maps for geoscientific models is set to be submitted later this year.

State of the Field

Exact HMC is not the only method for sampling distributions of this family. Two main alternatives include classical Hamiltonian Monte Carlo (Duane et al., 1987; Neal, 2011)

and Gibbs sampling with the Hit-and-Run Algorithm (Chen & Deely, 1992). HMC is a fast-mixing algorithm that is robust to high numbers of dimensions. However, it requires integrating equations of motion and using a Metropolis accept-reject step to account for numerical integration error. The numerical integration requires adjusting hyperparameters to balance state space exploration with a high acceptance probability (Hoffman et al., 2014). Gibbs sampling is a simpler method with no hyperparameters that always accepts samples, however, it can be slow to mix, particularly when constraints impose high correlation between variables. Since the constrained Gaussian HMC trajectories are analytically computable, Exact HMC enables the good mixing and high-dimensional capabilities of classical HMC with the always accepting and no hyperparameter properties of the Gibbs sampler. See Pakman & Paninski (2014) for a more detailed discussion of the differences between these methods. Other alternative methods include the R packages VeccTMVN (Cao & Katzfuss, 2024) and nntmvn (Cao & Katzfuss, 2025) which use Vecchia and nearest-neighbor approximations, respectively, to sample from a truncated multivariate Gaussian. However, these methods are both approximate and limited to sampling Gaussians with linear box constraints.

While there are some existing implementations of Exact HMC, to the best of our knowledge there are no existing Python implementations. Pakman & Paninski (2014) created an R implementation `tmg` and a Matlab implementation `hmc_tmg`; however, both are no longer maintained and the R package was archived from CRAN in 2021. Additionally, Bertolacci et al. (2024) partially implements the Exact HMC method in R but only for linear constraints.

Software Design

`tmg-hmc` is designed predominantly with two overarching principles: (1) ease of use, (2) performance optimization. As a sampling algorithm, `tmg-hmc` will predominantly be used as part of a much larger research pipeline. For ease of use, we created an object-oriented API based around the public facing `TMGSampler` class where end users can specify the unconstrained distribution, add constraints and then run the sampler. One important example of ease of use is how the sampler handles distributions with nonzero mean and non identity covariance. The Exact HMC algorithm requires a zero mean, identity covariance distribution so the sampler automatically rescales the input starting samples and adjusts the provided constraints to operate in the rescaled environment. Then, after sampling, the samples are transformed to their original scale. This way end users need not be familiar with the fine details of the sampling algorithm and can use `tmg-hmc` much the same way that they would sample from a `scipy.stats` distribution. Additionally, we minimized the number of dependencies in this package so that it won't conflict with any other required packages for other aspects of the research pipeline. For performance optimization, we implemented sparse-constraint and GPU-based accelerators that are controlled by the user through `sparse` and `gpu` flags in the initial class constructor. Additionally, we found that the three types of constraints — `LinearConstraint`, `SimpleQuadraticConstraint`, and `FullQuadraticConstraint` — have very different computational demands and hit-time calculation speeds. So, after rescaling the input constraints, the sampler class will automatically pick the simplest constraint type allowable for the rescaled constraint parameters. Finally, after profiling the original, all Python, implementation we found the most significant speed bottleneck to be the `FullQuadraticConstraint` hit time. So we optimized the implementation and rewrote it in C++ with Python bindings.

AI Usage Disclosure

We used Anthropic's Claude 3.7 Sonnet to optimize the C++ implementation of the Exact HMC hit times to quadratic constraints by removing redundant calculations. To do this, we initially solved the hit times analytically as shown in the Mathematica notebook located in `resources/HMC_exact_soln.nb` in the `tmg-hmc` repository. This resulted in a set of 8 solutions that were each a long mathematical expression taking over a page to write down. Then we used

119 Mathematica's CForm command to convert the expression to C. We then told Claude that the
 120 8 C functions represented a solution set and should follow a pattern and asked Claude to parse
 121 the functions for a pattern and rewrite them without performing any redundant calculations.
 122 By doing this we were able to simplify the ≈ 1000 lines of nested mathematical expressions in
 123 the original C implementation to a single ≈ 90 line function. We also saw a nearly 100 times
 124 speedup in the hit-time calculation from the original Python implementation after simplifying
 125 the expressions compared to an initial speedup of 5 times from just converting the Python
 126 implementation to C++.

127 To test the correctness of the generated code, we first tested that the code would compile. We
 128 then tested that the output hit times were close to the original C hit times from Mathematica
 129 within a numerical tolerance for a small set of test cases. Finally, we ran the sampler and
 130 tested that the sampled points reached and did not exceed the quadratic constraint bounds
 131 which is only possible if the hit times are correct. This led to about 15 iterations of testing the
 132 output code and asking Claude to refine its work because it was not correct. Then we finally
 133 got to a point where about half of the hit times appeared to be correct. This was a previously
 134 encountered issue with the Python implementation with a known solution. So we manually
 135 made this change to the C++ implementation which fixed the final issue and produced correct
 136 hit times in all of our tests.

137 For documentation writing, we used Claude to provide an initial Readme.md draft which
 138 was manually edited to ensure correctness. Additionally, we used Claude to create a custom
 139 generate docs workflow that would build the API_DOCS.md file from Python docstrings.
 140 We used a custom workflow so that we could control how custom types were displayed in the
 141 rendered documentation. We tested this for correctness by running the GitHub workflow and
 142 adjusting as needed.

143 Generative AI was not used for any aspect of paper writing.

144 Acknowledgments

145 This work was partially supported by the U.S. National Science Foundation under Grant
 146 No. DMS-2310632.

147 References

- 148 Agrell, C. (2019). Gaussian Processes with Linear Operator Inequality Constraints. *Journal of*
 149 *Machine Learning Research*, 20(135), 1–36.
- 150 Albert, J. H., & Chib, S. (1993). Bayesian analysis of binary and polychotomous response
 151 data. *Journal of the American Statistical Association*, 88(422), 669–679.
- 152 Bachoc, F., Lagnoux, A., & López-Lopera, A. F. (2019). Maximum likelihood estimation for
 153 Gaussian processes under inequality constraints. *Electronic Journal of Statistics*, 13(2),
 154 2921–2969.
- 155 Bachoc, F., López-Lopera, A. F., & Roustant, O. (2022). Sequential Construction and
 156 Dimension Reduction of Gaussian Processes Under Inequality Constraints. *SIAM Journal*
 157 *on Mathematics of Data Science*, 4(2), 772–800.
- 158 Bertolacci, M., Zammit-Mangion, A., Schuh, A., Bukosa, B., Fisher, J. A., Cao, Y., Kaushik,
 159 A., & Cressie, N. (2024). Inferring changes to the global carbon cycle with WOMBAT v2.0,
 160 a hierarchical flux-inversion framework. *The Annals of Applied Statistics*, 18(1), 303–327.
- 161 Cao, J., & Katzfuss, M. (2024). *Scalable Sampling of Truncated Multivariate Normals Using*
 162 *Sequential Nearest-Neighbor Approximation*. Preprint arXiv:2406.17307.

- 163 Cao, J., & Katzfuss, M. (2025). *Linear-Cost Vecchia Approximation of Multivariate Normal*
164 *Probabilities*. Preprint arXiv:2311.09426.
- 165 Chen, M.-H., & Deely, J. (1992). *Application of a new Gibbs hit-and-run sampler to a*
166 *constrained linear multiple regression problem*. Technical Report 92-21, Purdue University,
167 Center for Statistical Decision Sciences; Department of Statistics.
- 168 Cox, D. R., & Wermuth, N. (2002). On some models for multivariate binary variables parallel
169 in complexity with the multivariate Gaussian distribution. *Biometrika*, 462–469.
- 170 Da Veiga, S., & Marrel, A. (2012). Gaussian process modeling with inequality constraints.
171 *Annales de La Faculté Des Sciences de Toulouse : Mathématiques*, 21(3), 529–555.
- 172 Duane, S., Kennedy, A. D., Pendleton, B. J., & Roweth, D. (1987). Hybrid Monte Carlo.
173 *Physics Letters B*, 195(2), 216–222.
- 174 Emrich, L. J., & Piedmonte, M. R. (1991). A method for generating high-dimensional
175 multivariate binary variates. *The American Statistician*, 45(4), 302–304.
- 176 Gelfand, A. E., Smith, A. F., & Lee, T.-M. (1992). Bayesian analysis of constrained parameter
177 and truncated data problems using Gibbs sampling. *Journal of the American Statistical*
178 *Association*, 87(418), 523–532.
- 179 Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013).
180 *Bayesian data analysis*. CRC Press.
- 181 Hoffman, M. D., Gelman, A., & others. (2014). The No-U-Turn sampler: Adaptively setting
182 path lengths in Hamiltonian Monte Carlo. *J. Mach. Learn. Res.*, 15(1), 1593–1623.
- 183 López-Lopera, A. F., Bachoc, F., Durrande, N., & Roustant, O. (2018). Finite-Dimensional
184 Gaussian Approximation with Linear Inequality Constraints. *SIAM/ASA Journal on*
185 *Uncertainty Quantification*, 6(3), 1224–1255.
- 186 Maatouk, H. (2017). *Finite-dimensional approximation of Gaussian processes with inequality*
187 *constraints*. Preprint arXiv:1706.02178.
- 188 Neal, R. M. (2011). MCMC using Hamiltonian dynamics. *Handbook of Markov Chain Monte*
189 *Carlo*, 2(11), 2.
- 190 Neelon, B., & Dunson, D. B. (2004). Bayesian isotonic regression and trend analysis.
191 *Biometrics*, 60(2), 398–406.
- 192 Pakman, A., & Paninski, L. (2014). Exact Hamiltonian Monte Carlo for truncated multivariate
193 Gaussians. *Journal of Computational and Graphical Statistics*, 23(2), 518–542.
- 194 Paninski, L., Simoncelli, E., & Pillow, J. (2003). Maximum likelihood estimation of a stochastic
195 integrate-and-fire neural model. *Advances in Neural Information Processing Systems*, 16.
- 196 Polson, N. G., Scott, J. G., & Windle, J. (2014). The bayesian bridge. *Journal of the Royal*
197 *Statistical Society Series B: Statistical Methodology*, 76(4), 713–733.
- 198 Robert, C. P., & Casella, G. (1999). *Monte Carlo statistical methods* (Vol. 2). Springer.
- 199 Stanley, M., Batlle, P., Patil, P., Owhadi, H., & Kuusela, M. (2025). Confidence intervals for
200 functionals in constrained inverse problems via data-adaptive sampling-based calibration.
201 *Preprint arXiv:2502.02674*.
- 202 Swiler, L. P., Gulian, M., Frankel, A. L., Safta, C., & Jakeman, J. D. (2020). A Survey of
203 Constrained Gaussian Process Regression: Approaches and Implementation Challenges.
204 *Journal of Machine Learning for Modeling and Computing*, 1(2).
- 205 Tobin, J. (1958). Estimation of relationships for limited dependent variables. *Econometrica:*
206 *Journal of the Econometric Society*, 24–36.